2026-08-10 時点のまとめ（印刷して読む用）

これ 1 枚で完結する。 他のファイルを開かなくても読めるよう、数字は本文に
書き写してある。読み終えたときに「次に何を決めるべきか」が分かる状態を目指した。

1. いまどこにいるか

	値
最新の本番スコア	**0.1409**
トップ	0.6396
配布学習例（参考値）	0.0633
モデル提出の締切	**8/14（金）23:59**
レポート提出の締切	8/17（月）23:59
残りの提出回数	**2 回**
追加学習	**まだしていない**（素の重みのまま）

スコアの推移。**すべて素の重みで、推論時のつまみを変えただけである。**

回	設定	スコア	合計時間
1	素のまま（ `n_action_steps=50` ）	0.05832	147.6 秒
2	`n_action_steps=10`	0.096	220.5 秒
3	`n_action_steps=5` ＋ `EMA=0.3`	**0.1409**	305.4 秒

2.4 倍になったが、学習は一度もしていない。

2. 用語（最小限）

語	意味
タスク	「黒いボウルを皿に置く」など。元は 40 種
タスク変種	タスクに摂動を掛けたもの。全 10,030 個

語	意味
摂動	見た目や初期状態を変える操作。7 カテゴリある
エピソード	1 回の試行。ロボットが動いて成功か失敗かが決まる
`n_action_steps`	1 回の推論で作った 50 歩分の計画のうち、**何歩実行してから作り直すか**
EMA	出した動きに直前の動きを混ぜてならす処理
チャンク	1 回の推論が作る 50 歩分の計画

3. 今日いちばん重要な発見：本番の中身が分かった

採点ログには `/reset`（エピソードの開始）と `/act`（1 歩の指示）が**すべて記録されている**。並び順を数えると、1 エピソードずつの歩数が出る。

```
n=10  →  300, 300, 300, 300, 128, 164, 300, 186   スコア 0.096
n=5   →  300, 300, 300, 300, 138, 132, 174, 182   スコア 0.141
```

ここから 3 つ分かる。

① 本番の打ち切りは 300 歩である

配布キットの既定は 600 歩。**私たちは倍の時間を与えて測っていた。**
つまり手元の成功率は甘い。

手元で測ったタスク別の所要歩数を、300 歩の物差しで見るとこうなる。

タスク	成功までの平均歩数	300 歩で間に合うか
コンロにボウルを置く	89.7	余裕
引き出しのボウルを皿へ	132.7	余裕
ミルクをかごへ	171.4	余裕
トマトソースをかごへ	**248.3**	**半分近く落ちる**

「速く終える」ことが、これまで思っていたより重要だった。

② 本番は 8 エピソードしか回さない

`/reset` が 8 回しか出ていない。**1 本の成否がスコアを 0.03〜0.05 動かす。**

リーダーボードの小さな差は、統計的にはほとんど意味を持たない。
逆に言えば、**運が悪いだけで順位は簡単に上下する。**

③ スコアの式が推定できた

2 回分のスコアと `done` の本数から、次で説明がつく。

```
スコア = （成功した本数 / 8） × 指標の係数

  0.096 = (2/8) × 0.384
  0.141 = (3/8) × 0.376        → 係数はどちらも約 0.38
```

`done` （タスクを完了）した本数はログから 3 本 → 4 本と確定している。
成功が 2 → 3 なら、**どちらの回も 1 本は衝突で失格していた**ことになる。

係数 0.38 の意味は大きい。 滑らかさ・軌道の長さ・所要歩数といった
配点で、**満点の 38% しか取れていない。**

4. スコアを上げる道は 2 本ある

いまは 0.1409 ＝（3/8）× 0.38。トップは 0.6396。

道	いま	もし改善したら
A. 成功本数を増やす	3 / 8	5/8 なら 0.235、7/8 なら 0.33
B. 指標の係数を上げる	0.38	0.6 なら 0.22、0.8 なら 0.30

片方だけではトップに届かない。 0.6396 に必要なのは、たとえば
「7/8 成功 × 係数 0.73」である。**両方を同時に上げる必要がある。**

A（成功本数）を上げる手

- **追加学習**。まだ一度もしていない。いちばん大きな未使用の手
- `n_action_steps` をさらに縮める（5 → 2）
- 衝突を減らす（毎回 1 本は衝突で落ちている）

B（係数）を上げる手

配点に入っているのは、分かっている範囲でこれだけある。

指標	中身	上げ方
`time`	**歩数**（壁時計ではない）	**速く終える**

指標	中身	上げ方
`cartesian_path_length`	手先が動いた距離	遠回りしない
`jerk` （4 種類）	動きのガクつき	なめらかにする
`sparc`	動きの滑らかさの別指標	同上

ここに重要な緊張関係がある。

> **EMA は jerk を 22% 下げたが、成功までの歩数を 147.5 → 160.5 に増やした。**
> 滑らかさで得点し、速さで失点している。どちらの配点が重いかは非公開。

5. 見落としていた「ほぼ無料」の手

`time` は壁時計ではなく歩数で測られる（運営回答で確定）。
そして 1 時間の制限に対して、いまは 305 秒しか使っていない。

つまり **推論をどれだけ重くしても、スコア上は無料である。**

設定	予測される合計時間	上限 3,600 秒に対して
いま（n=5）	305 秒	8.5%
n=2	585 秒	16%
n=1	1,041 秒	29%
n=5 ＋ 3 回サンプリング	676 秒	19%
n=2 ＋ 3 回サンプリング	**1,496 秒**	**42%**

まだ半分以上余っている。 「推論が重くなるから」を理由に手を控える
必要はまったくない。

（3 回サンプリング＝同じ場面で計画を 3 回引き、**互いに最も近い 1 本を選ぶ**。
外れを引く確率を下げる手。平均ではなく実在の 1 本を選ぶのは、
平均するとハンドの開閉が中間値になって物を掴めなくなるため。）

6. 手元の評価が本番を予測できない問題

これがいちばん厄介である。

	手元（配布された 4 タスク）	本番
`n_action_steps=50`	成功率 0.0625	0.058
`n_action_steps=10`	成功率 **0.875**	**0.096**

最初の回で近かったのは**偶然**だった。どちらもほぼ全滅の水準なので、
何を測っても 0.05〜0.07 に落ちる。**一致に情報量が無かった。**
私はこれを「手元の評価は本番を再現する」と判断したが、**誤りだった。**

原因は配布リポジトリにそのまま書いてある。

> 本番評価は**非公開タスクを含む別のタスクセット**で実施される。

配布された 4 タスクは **example** であって、採点対象そのものではない。

対処：検証用スイートを作った

LIBERO-plus の 10,030 変種から、example 以外を **84 タスク**選んで測る。
7 つの摂動カテゴリを全部入れ、**カテゴリ別に成功率を出す。**

当初は「example 4 件が視覚系の摂動だから本番もそこ」と読んで 2 カテゴリに
絞っていたが、**4 サンプルからの推測でしかない**と考え直して全部入れた。
外れていたときに気づけないほうが困る。

副産物として、どのカテゴリが弱いかが分かる。 そこが学習で厚くすべき場所になる。

7. 決めていただきたいこと

① 運営 Q&A No.24 の催促（いちばん急ぐ）

> 予選の最終評価には、複数回提出した場合どの提出物が用いられますか。
> (a) 最後に採点が完了した提出　(b) スコアが最も高い提出　… **対応中**

残り 2 回の使い方が変わる。

- **(a) なら**：最後の 1 回は「その時点で最良と分かっている構成」を出し直す。
  つまり実験に使えるのは実質 1 回だけ
- **(b) なら**：2 回とも実験に使える

未回答のままなら (a) と仮定して動くしかない。

② Ubuntu の GPU マシンを手配するか

学習は長時間の連続実行が要る。Colab はセッション上限で切れる（実際 9,100 歩で切れた）。**GPU マシンが取れるなら学習はそちらへ回したい。**
手順書は用意済みで、コマンド 3 つで回る。

③ 残り 2 回を何に使うか

いま考えている候補。

候補	見込み	根拠
`n_action_steps=2` ＋ EMA	中	50→10→5 と単調に良くなっている
3 回サンプリングを足す	中	時間は無料。外れを引く確率が下がる
追加学習した重み	**大きいが未知**	まだ一度も試していない

追加学習が間に合うかどうかで組み立てが変わる。 ②の答え次第。

8. 移動から戻ったら最初にやること

1. **Colab で検証用スイートを回す**（`MAX_STEPS=300` で）
   - `n=10` ・EMA なし（本番 0.096 と対応）
   - `n=5` ・EMA 0.3（本番 0.1409 と対応）
   - **2 点あれば、手元の指標が本番を追えるか判定できる**
2. カテゴリ別の成功率を見て、**どの摂動が弱いか**を確認する
3. その結果を見て、学習データの選び方を決める

所要 80 分。手順は `solution/COLAB_EVAL.md` にある。

9. 私が間違えて、あとで撤回した判断

同じ間違いを繰り返さないための記録。 私の言うことを鵜呑みにしない材料としても使ってください。

私の主張	実際	なぜ間違えたか
手元の評価は本番スコアを再現する	**しない**（0.875 対 0.096）	2 つの数字が近かったのを根拠にしたが、どちらもほぼ全滅の水準で情報量が無かった
`time` は壁時計で測られる	**歩数だった**	運営回答を読む前に推測した

私の主張	実際	なぜ間違えたか
チャンクの継ぎ目を均す処理は不要	**n を縮めたら必要になった**	継ぎ目が 5 倍に増える前提を考えていなかった
学習の優先度は低い（手元で 0.875 だから）	**最優先に戻した**	その 0.875 が example での数字だった
`n_action_steps` を縮めると 1 時間制限に当たる	**n=1 でも 29%**	実測せずに見積もった
評価が止まって見えるが待ってよい	**ログの設定の問題だった**	出力が無いことの原因を調べずに答えた

共通しているのは「測る前に答えた」ことである。 以降は、判断の前に
必ず測るか、一次情報（配布コード・運営回答・ログ）を確認するようにしている。

10. 手元にある資料の場所

印刷しないが、戻ってから見るもの。

知りたいこと	ファイル
まだ手をつけていないこと	`docs/talk/open_items.md`
測った数字すべて	`docs/strategy/05_実測結果.md`
用語と全体像	`docs/strategy/README.md`
評価の回し方	`solution/COLAB_EVAL.md`
学習の回し方	`docs/howto/01_retrain.md`
提出物の作り方	`docs/howto/02_build_submission.md`
本番の採点ログ	`docs/prod_eval_results/`